

TP Intégratif PHP

Gestionnaire de Bibliothèque Personnelle

Contexte : Vous développez une application PHP native de gestion d'une bibliothèque personnelle. Les questions sont progressives — testez votre code dans le navigateur à chaque étape.

L'application est composée des fichiers suivants :

Fichier	Rôle
index.php	Page d'accueil - liste tous les livres
ajouter.php	Formulaire d'ajout d'un nouveau livre
traitement.php	Validation et enregistrement du livre
recherche.php	Recherche par titre ou auteur
fonctions.php	Fonctions utilitaires (inclus partout avec <code>require_once</code>)
login.php	Formulaire de connexion
logout.php	Déconnexion et destruction de session
livres.csv	Fichier CSV de persistance des données

Partie 1 – Tableaux et Fonctions

Question 1 – Structure des données et affichage

Chaque livre est représenté par un tableau associatif avec les champs : titre (string, obligatoire), auteur (string, obligatoire), annee (integer, entre 1000 et l'année en cours), genre (string : Roman / Science-fiction / Histoire / Autre), note (float, de 0 à 5), lu (boolean), date_ajout (string généré avec `date('d/m/Y')`).

- Dans `index.php`, déclarez `$genres` et créez un tableau `$livres` avec 3 livres pré-remplis.
- Affichez dans une balise `<pre>` le contenu du premier livre avec `var_dump()`.
- Affichez tous les livres dans un tableau HTML : Titre | Auteur | Année | Genre | Note | Lu | Actions. Ajoutez dans Actions un lien Voir vers `details.php?id=X`.
- Affichez en haut de page : nombre total de livres, nombre de livres lus, moyenne des notes.

Question 2 – Fonctions utilitaires (fonctions.php)

Créez `fonctions.php` et codez les fonctions suivantes :

- `trierLivres(array $livres, string $critere, string $ordre = 'asc') : array` — trie par 'titre', 'auteur', 'annee' ou 'note' avec `usort()`.
 - `filtrerParGenre(array $livres, string $genre) : array` — retourne les livres du genre donné.
 - `calculerMoyenne(array $livres) : float` — moyenne des notes des livres dont la note > 0.
- Incluez `fonctions.php` dans `index.php` et appliquez `trierLivres()` pour afficher les livres triés par titre.
 - Affichez la moyenne calculée par `calculerMoyenne()` en haut de la page.

Partie 2 – Formulaires et Validation

Question 3 – Ajout d'un livre

Créez `ajouter.php` : formulaire POST vers `traitement.php` avec les champs titre, auteur, annee (number), genre (select), note (number step=0.5), lu (checkbox).

Dans traitement.php :

7. Refusez toute requête qui n'est pas POST (redirigez vers index.php).
8. Nettoyez les données avec trim() et htmlspecialchars(). Validez chaque champ et collectez toutes les erreurs dans \$erreurs.
9. En cas d'erreur : stockez erreurs et données dans la session, redirigez vers ajouter.php. En cas de succès : sauvegardez le livre, redirigez vers index.php avec un message de confirmation.

N.B. : Rendez le formulaire sticky : réaffichez les valeurs de \$_SESSION['form_data'] et les erreurs en cas d'échec.

Question 4 – Recherche

Créez recherche.php : formulaire GET avec un champ texte (mot-clé) et une liste déroulante de genre (+ option « Tous »).

10. Filtrez les livres : cherchez le mot-clé dans le titre ET l'auteur avec strpos() (insensible à la casse), puis appliquez le filtre de genre si besoin.
11. Affichez les résultats dans un tableau HTML. Si aucun résultat, affichez un message clair.
12. Conservez les valeurs saisies dans le formulaire entre les soumissions (sticky form avec \$_GET).

Partie 3 – Persistance avec les Fichiers

Question 5 – Lecture / écriture CSV

Le fichier livres.csv stocke un livre par ligne : titre;auteur;annee;genre;note;lu;date_ajout

Dans fonctions.php, ajoutez :

- chargerLivres(string \$fichier) : array — lit chaque ligne avec fgetcsv() (séparateur ';'). Retourne [] si le fichier n'existe pas.
- sauvegarderLivre(string \$fichier, array \$livre) : void — ajoute une ligne en fin de fichier avec fputcsv() en mode 'a'.

13. Créez manuellement livres.csv avec 2 lignes de test. Remplacez \$livres par chargerLivres() dans index.php.
14. Dans traitement.php, remplacez le stockage en session par sauvegarderLivre(). Testez un ajout complet.
15. Créez details.php : récupérez l'id via \$_GET['id'], affichez le livre correspondant. Si l'id est invalide, redirigez vers index.php.

Question 6 – Compteur de visites

16. Dans index.php, lisez le fichier stats.txt avec file_get_contents(), incrémentez la valeur, sauvegardez avec file_put_contents() et affichez le compteur en bas de page.

Partie 4 – Sessions et Cookies

Question 7 – Authentification par session

Identifiants codés en dur : identifiant = etudiant, mot de passe = sinfo2026.

17. Créez login.php : formulaire POST. Vérifiez les credentials. Si corrects, stockez \$_SESSION['connecte'] = true et \$_SESSION['login'], redirigez vers index.php. Sinon, affichez un message d'erreur.
18. Protégez toutes les pages : vérifiez \$_SESSION['connecte'] en début de chaque fichier, sinon redirigez vers login.php.
19. Dans logout.php, appelez session_unset() + session_destroy() puis redirigez vers login.php.
20. Affichez le login et un lien Déconnexion dans l'en-tête de chaque page.

Question 8 – Préférences par cookie

Créez preferences.php : formulaire POST avec deux choix : critère de tri (Titre / Auteur / Année / Note) et nombre de livres par page (5 / 10 / 20).

21. Enregistrez chaque préférence dans un cookie séparé valable 30 jours avec setcookie().
22. Dans index.php, lisez les cookies avec \$_COOKIE, appliquez le tri avec trierLivres() et la pagination avec array_slice(). Valeurs par défaut si aucun cookie : tri par Titre, 10 livres par page.
23. Le formulaire pré-sélectionne les valeurs actuelles des cookies.